

BUT 2 Informatique

2025 — 2026



Rapport de Stage

Reprise et amélioration d'une plateforme web
de mise en relation dans le secteur du BTP



MARCHÉS BTP

Marchés BTP

Incubateur IMT Business School
9, rue Charles Fourier — 91011 Évry-Courcouronnes

Auteur : Mohammed BEQQAL AZIRAR

Maître de stage : Marko MARINKOVIC

Encadrante technique : Nadia AISSANI

Enseignant référent : Lucian FINTA

Remerciements

Avant de détailler le contenu de mon stage, je souhaite remercier les personnes qui ont contribué à cette expérience, inoubliable et enrichissante au sein de Marchés BTP.

Je remercie tout d'abord Marko Marinkovic, fondateur de Marchés BTP et mon maître de stage, pour la confiance qu'il m'a accordée dès le premier jour. Il m'a laissé une grande autonomie dans mon travail tout en restant disponible pour répondre à mes questions et orienter mes choix techniques. Sa vision du projet et son enthousiasme m'ont motivé tout au long de ces huit semaines.

Un grand merci à Nadia Aissani, développeuse Full Stack et encadrante technique, qui m'a accompagnée au quotidien. Elle a pris le temps de m'expliquer l'architecture du projet, de me montrer des techniques que je ne connaissais pas encore et de relire mon travail. Sa patience, sa pédagogie et sa disponibilité ont été déterminantes dans ma progression. C'est grâce à son encadrement que j'ai pu gagner en autonomie et en confiance sur les aspects techniques du projet.

Je remercie aussi Ayman et Abraham pour leur aide et les échanges techniques que nous avons eus, ainsi que Melissa pour son travail sur la communication du projet et Cemilia pour ses éclairages sur les aspects juridiques et financiers.

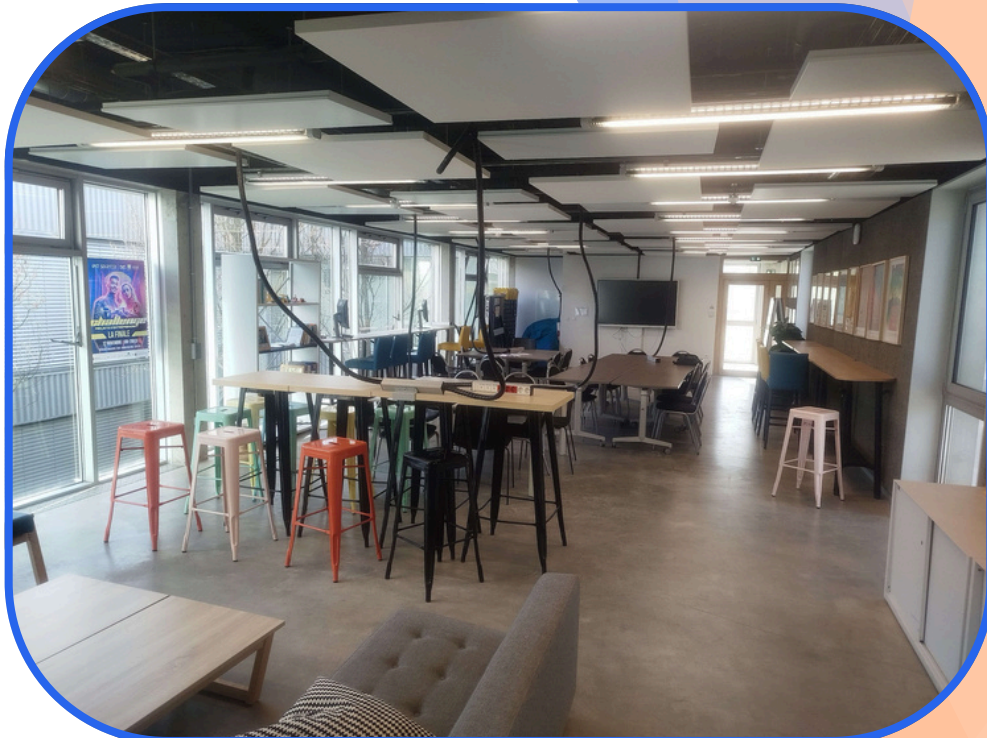
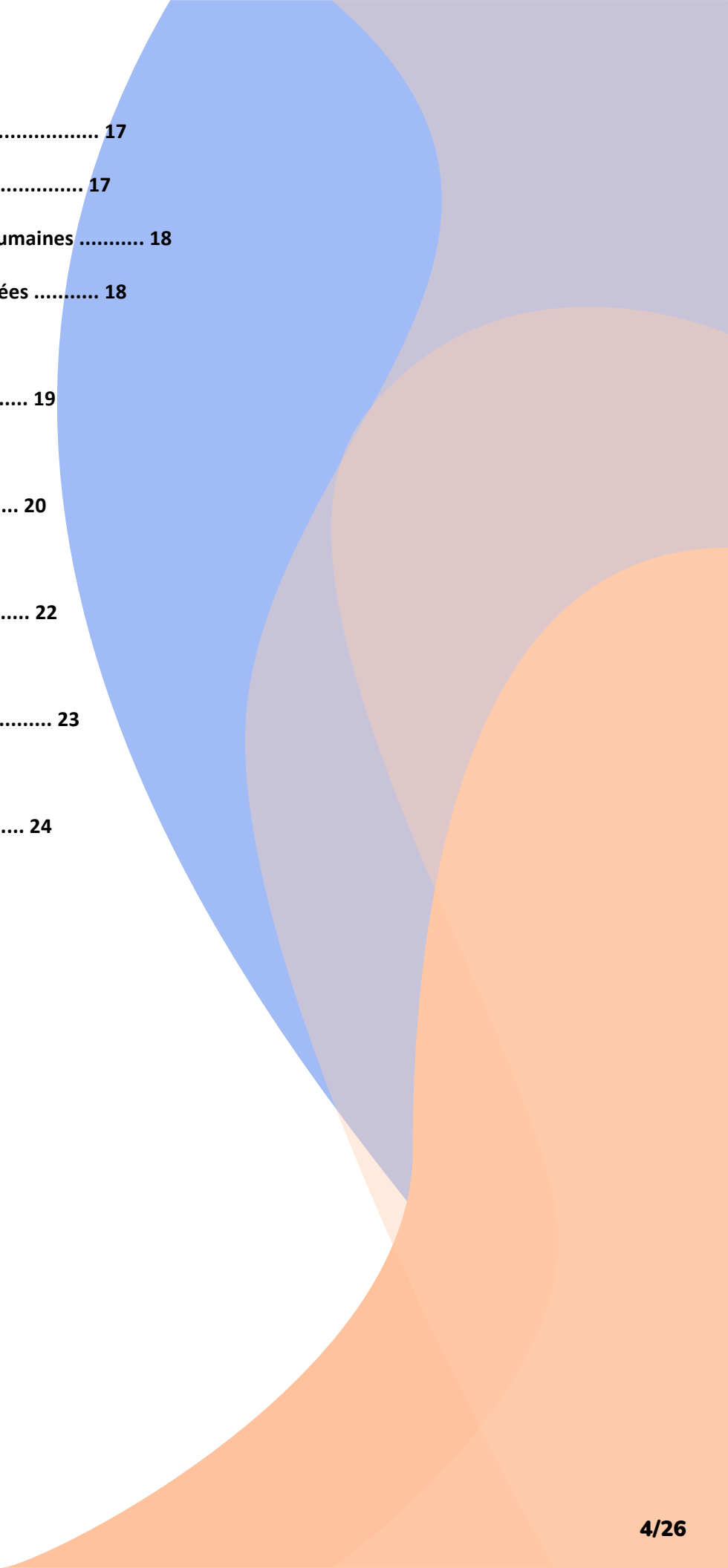


Figure 1 : Espace de coworking de l'incubateur IMT Business School

Sommaire

Remerciements	2
Introduction	4
1. Présentation de l'entreprise	5
1.1 Marchés BTP : une startup dans le numérique du BTP .	5
1.2 L'incubateur IMT Business School	5
1.3 L'équipe et l'organisation	6
2. Environnement de travail et cadrage technique	7
2.1 Mon intégration et le cadre quotidien	7
2.2 Découverte du projet existant	7
2.3 Architecture et technologies	8
2.4 Méthodologie de travail	9
3. Réalisations techniques	9
3.1 Restructuration de l'architecture backend	9
3.2 Développement de nouvelles fonctionnalités backend ..	10
3.3 Développement frontend : services et composants	12
3.4 Adaptation responsive des interfaces	13
3.5 Documentation Swagger de l'API REST	14
3.6 Tests fonctionnels et rapport de bugs	16
3.7 Passation du projet	16



4. Bilan et compétences acquises	17
4.1 Compétences techniques	17
4.2 Compétences professionnelles et humaines	18
4.3 Difficultés rencontrées et leçons tirées	18
 Conclusion	 19
 Glossaire	 20
 Références	 22
 Table des illustrations	 23
 Annexes	 24

Introduction

Actuellement en deuxième année de BUT Informatique à l'IUT de Villetaneuse, situé à l'Université Sorbonne Paris Nord, j'ai effectué mon stage du 26 janvier au 20 mars 2026 au sein de Marchés BTP, une startup incubée à l'IMT Business School d'Évry-Courcouronnes. Ce stage de huit semaines représente ma première expérience professionnelle dans le développement web et même en général, et il constituait une occasion de confronter les connaissances acquises à l'IUT à la réalité d'un projet concret dans un cadre professionnelle.

Marchés BTP développe une plateforme en ligne destinée à mettre en relation les entreprises du bâtiment et travaux publics, les freelances du secteur et les porteurs d'appels d'offres. À mon arrivée, la plateforme existait déjà dans une version développée par des intervenants précédents, mais son code présentait de nombreux problèmes : une architecture peu claire, mal structuré, des fonctionnalités incomplètes, des erreurs récurrentes et aucune documentation technique. Mon rôle a été de reprendre ce projet, de le restructurer et de le faire évoluer.

Mes missions principales ont consisté à analyser le code existant, à proposer et mettre en place une architecture MVC plus rigoureuse, à développer de nouvelles fonctionnalités côté backend et frontend, à adapter l'interface pour les écrans mobiles, à rédiger la documentation complète de l'API avec Swagger, et enfin à produire un rapport de tests identifiant les bugs restants. Ce travail m'a amené à utiliser des technologies variées: React et Tailwind CSS côté frontend, Node.js et Express côté backend, PostgreSQL et Prisma pour la base de données, ainsi que Postman et Swagger pour les tests et la documentation.

Au fil de ce rapport, je me suis demandé comment reprendre un projet développé par d'autres et le faire évoluer de manière cohérente, tout en acquérant de nouvelles compétences techniques. Cette question m'a guidé dans l'ensemble de mes choix pendant le stage.

Dans un premier temps, je présenterai l'entreprise Marchés BTP et son environnement. Je décrirai ensuite le cadre de travail et l'état du projet à mon arrivée. La troisième partie, qui constitue le cœur de ce rapport, détaillera mes réalisations techniques semaine par semaine. Enfin, je dresserai un bilan des compétences acquises et de cette expérience professionnelle.

Présentation de l'entreprise

Marchés BTP : une startup dans le numérique du BTP

Le secteur du bâtiment et des travaux publics en France représente un poids économique considérable, avec plusieurs centaines de milliers d'entreprises allant du micro-entrepreneur à de grands groupes. Pourtant, la mise en relation entre ces acteurs reste souvent informelle : le bouche-à-oreille, les réseaux personnels et les appels téléphoniques constituent encore les principaux canaux pour trouver un prestataire ou répondre à un appel d'offres mais aussi des contrats hors la loi. C'est dans ce contexte que Marchés BTP a vu le jour.

Marchés BTP est une société par actions simplifiée (SAS) fondée par Marko Marinkovic et Brahim. Leur idée : créer une plateforme numérique qui centralise les appels d'offres du BTP et facilite la mise en relation entre trois types d'utilisateurs. D'un côté, les entreprises du bâtiment qui cherchent des prestataires pour leurs chantiers. De l'autre, les freelances et auto-entrepreneurs qui proposent leurs compétences (plomberie, électricité, maçonnerie, peinture, etc). Et au milieu, les porteurs d'appels d'offres qui publient leurs besoins et reçoivent des candidatures.

La plateforme permet de gérer l'ensemble du cycle : publication d'un appel d'offres, candidature des freelances, sélection par l'entreprise, création d'un contrat, suivi des étapes (milestones), messagerie entre les parties, paiements et évaluations mutuelles. L'entreprise était encore en phase de développement au moment de mon stage : le produit n'était pas encore commercialisé, mais les fondateurs travaillaient activement à le rendre fonctionnel et présentable pour d'éventuels investisseurs.

L'incubateur IMT Business School

Marchés BTP est hébergée au sein de l'incubateur de l'IMT Business School, situé au 9, rue Charles Fourier à Évry-Courcouronnes 91000. Cet incubateur accompagne des startups en phase de lancement en leur offrant un espace de travail partagé, un accès à un réseau d'experts, ainsi que des ressources pour structurer leur projet. Pour Marchés BTP, cet hébergement représentait un avantage important : un cadre professionnel sans les coûts d'un bureau propre, et la possibilité d'échanger avec d'autres porteurs de projets comme Nassim créateur de deux SaaS dont Visulard.

Concrètement, nous travaillions dans un espace de coworking où chaque personne avait sa place autour d'une grande table. L'ambiance était plutôt décontractée, loin du formalisme que j'imaginais en entreprise. On se tutoyait, les échanges étaient directs, et il suffisait de se tourner vers son voisin pour poser une question. Ce cadre m'a beaucoup plu : j'ai ressenti dès le début une vraie proximité avec l'équipe et une certaine liberté de travail.

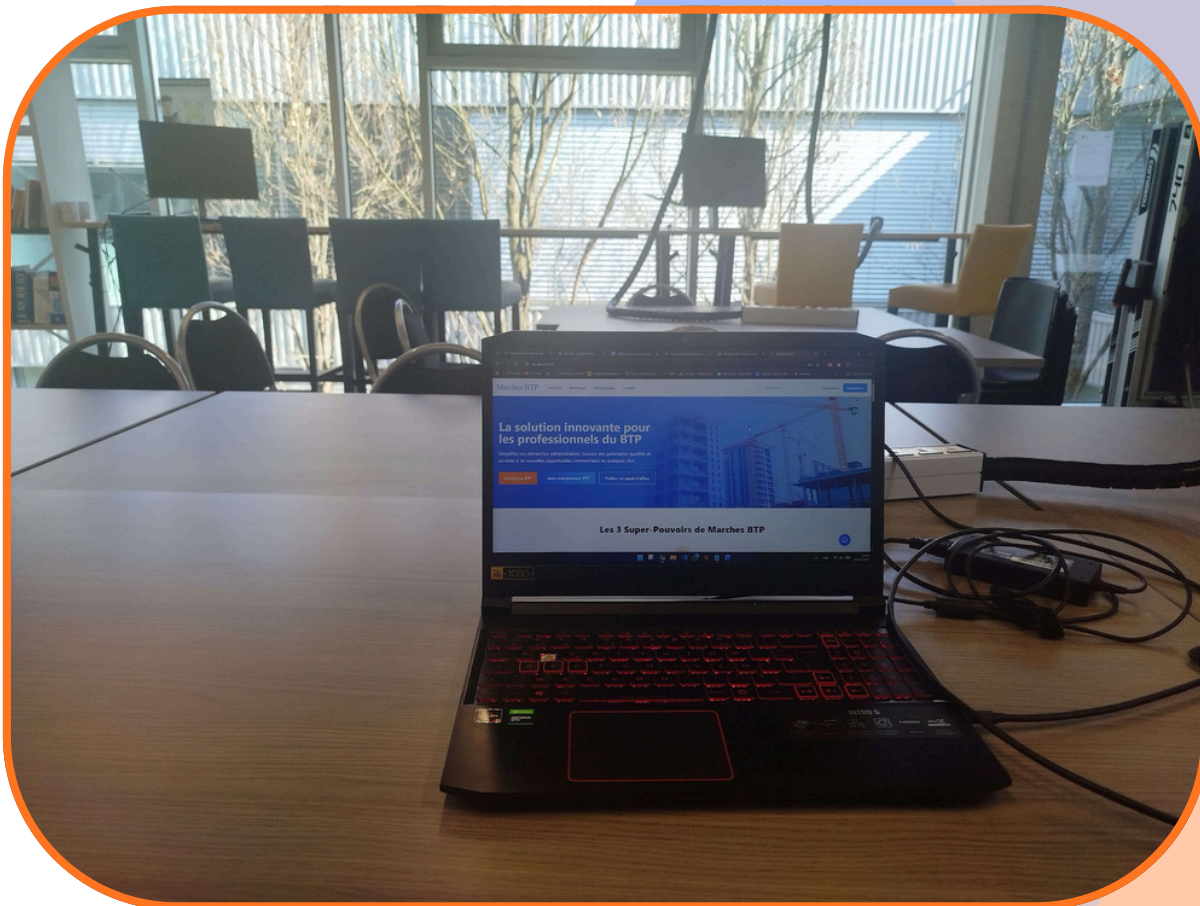


Figure 2 : Poste de travail avec la plateforme Marchés BTP ouverte

L'équipe et l'organisation

L'équipe de Marchés BTP était composée d'une dizaine de personnes aux profils variés. Marko, le fondateur, pilotait la vision produit et les relations avec l'incubateur. Brahim, co-fondateur, s'occupait davantage du volet commercial et métier. Côté technique, Nadia Aissani, développeuse Full Stack, supervisait le développement au quotidien. Ayman et Abraham travaillaient également sur le développement. Melissa gérant la communication et le contenu de la plateforme. Cemilia, juriste financière, intervenait sur les questions légales et contractuelles. Un juriste supplémentaire était présent une semaine sur deux qui était Frank qui travailla la plupart du temps avec Cemilia.

Cette organisation, typique d'une startup, m'a surpris par rapport à ce que j'avais pu imaginer d'une entreprise classique. Il n'y avait pas de hiérarchie lourde ni de processus formels : chacun connaissait ses tâches, les discussions se faisaient naturellement, et les décisions se prenaient rapidement. Pour moi qui découvrais le monde professionnel, c'était un premier contact rassurant : j'ai pu poser des questions sans hésiter et proposer des idées sans craindre qu'elles soient ignorées.

Environnement de travail et cadrage technique

Mon intégration et le cadre quotidien

Mon premier jour de stage a débuté par une présentation générale de l'entreprise. Marko et Brahim nous ont montré une vidéo réalisée par Melissa pour expliquer le fonctionnement de la plateforme et son positionnement sur le marché. Après le visionnage, chacun a donné son avis. J'ai suggéré que la voix artificielle rendait le message moins engageant et qu'une voix humaine serait plus adaptée pour mieux attirer le potentiel client. J'ai aussi remarqué que les couleurs de la vidéo étaient trop variées et proposé de reprendre celles du logo Marchés BTP, à savoir le bleu et le orange, pour garder une cohérence visuelle. Marko et Brahim ont noté ces remarques sur un tableau, ce qui m'a fait comprendre que mon avis comptait dès le premier jour.

Le rythme de travail s'est vite installé : en présentiel à l'incubateur trois à quatre jours par semaine, et en télétravail le reste du temps, généralement le vendredi. En télétravail, on communiquait par Discord pour

les appels et par WhatsApp pour les échanges rapides. En présentiel, on était tous autour de la même table avec nos propres ordinateurs portables : pour poser une question, il suffisait de se tourner vers la personne concernée. J'ai préféré le présentiel au télétravail, car les problèmes se résolvaient beaucoup plus vite en face à face puisqu'il n'y a pas de délai de réponse.

Découverte du projet existant

Dès le deuxième jour, Nadia m'a présenté l'état du projet. La plateforme reposait sur trois serveurs distincts : un serveur frontend sur le port 3000, un serveur backend principal sur le port 3001, et un second serveur backend sur le port 3002 qui avait été ajouté pour gérer certaines fonctionnalités spécifiques, notamment pour les dashboards entreprise et administrateur. Le problème, c'est que personne ne savait vraiment pourquoi ce deuxième backend existait : il n'y avait aucune documentation qui justifiait cette séparation.

En analysant le site, le constat était assez sévère. L'interface s'affichait correctement, mais beaucoup de fonctionnalités ne marchaient pas. Les notifications généraient un message d'erreur à chaque ouverture. L'onglet géolocalisation, prévu pour permettre aux entreprises de trouver des prestataires autour d'elles, ne fonctionnait pas du tout. Les appels d'offres provoquaient des erreurs réseau. Et ces problèmes se retrouvaient dans tous les dashboards. Nadia m'a expliqué que la cause principale était la désorganisation du code : les routes étaient éparpillées entre les deux backends, les fichiers étaient mal nommés, et il n'y avait ni commentaires ni séparation claire des responsabilités.

Le troisième jour, j'ai installé l'environnement de développement sur mon ordinateur personnel : Node.js pour le serveur, PostgreSQL pour la base de données, et Visual Studio Code comme éditeur. J'ai rencontré un premier problème : PostgreSQL ne se lançait pas correctement parce que je l'avais installé sur un disque différent de celui du système. Après avoir déplacé l'installation, tout a fonctionné. Nadia m'a montré comment ouvrir deux terminaux dans VS Code : l'un pour le backend, l'autre pour le frontend. Ce n'était pas intuitif au départ, d'autant que les dossiers étaient mal nommés — « auth.api » pour le backend au lieu d'un nom plus explicite.

Architecture et technologies

La plateforme utilise une architecture séparée en frontend et backend. Le frontend est développé en React, une bibliothèque JavaScript que j'avais découverte en cours mais jamais utilisée sur un projet de cette envergure. Le style repose sur Tailwind CSS, un framework qui fonctionne avec des classes utilitaires appliquées directement dans le HTML. Le backend est construit avec Node.js et le framework Express, et communique avec une base de données PostgreSQL via l'ORM Prisma. L'authentification est gérée par des tokens JWT (JSON Web Tokens).

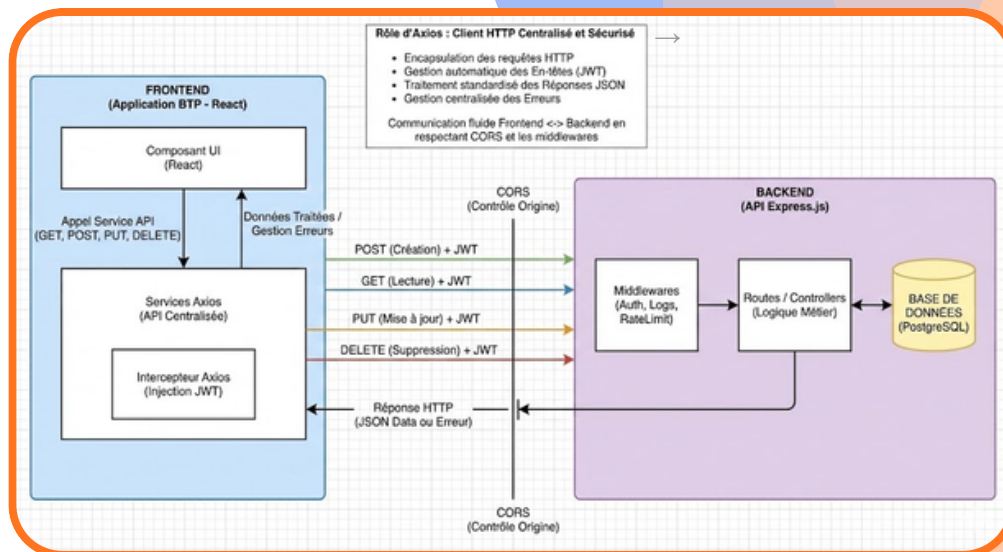


Figure 3 — Architecture technique de la plateforme Marchés BTP

Le projet comptait, à mon arrivée, une centaine de composants React côté frontend et une trentaine de fichiers côté backend (routes, services, contrôleurs). La base de données était décrite dans dix-neuf fichiers Prisma contenant vingt-et-un modèles (utilisateurs, entreprises, freelances, appels d'offres, contrats, etc). Plusieurs de ces modèles existaient dans le schéma mais n'étaient pas encore exploités par le code applicatif.

Méthodologie de travail

Nous n'utilisons pas de méthodologie agile formelle avec des sprints ou des stand-ups quotidiens. Le fonctionnement était plus souple : des points d'équipe deux à trois fois par semaine, au cours desquels chacun présentait ce qu'il avait fait et ce qu'il prévoyait. Les tâches étaient réparties naturellement selon les compétences de chacun, et on s'ajustait en fonction des urgences. Marko et Brahim voulaient initialement un suivi quotidien par WhatsApp, mais Nadia a proposé un suivi plus structuré sur un outil de gestion de tâches, ce que tout le monde a accepté. Cela m'a permis de mieux planifier mon travail et de visualiser l'avancement global du projet.

Réalisations techniques

Cette partie décrit les principales missions que j'ai réalisées pendant mes huit semaines de stage. Plutôt que de raconter le stage au jour le jour, j'ai choisi de regrouper mes réalisations par thème, en les mettant en perspective avec les compétences du BUT Informatique.

Restructuration de l'architecture backend

La première semaine a été essentiellement consacrée à l'analyse du code existant. Après avoir passé plusieurs jours à lire les fichiers, j'ai identifié un problème structurel majeur : le code mélangeait les responsabilités dans les mêmes fichiers, sans séparation claire entre le traitement des requêtes HTTP, la logique métier et l'accès à la base de données. Un même fichier pouvait contenir à la fois la définition des routes, les validations, les requêtes Prisma et la mise en forme des réponses.

Deux approches ont été évoquées en équipe : une architecture micro-services et une architecture MVC.

J'ai proposé le modèle MVC (Modèle – Vue – Contrôleur) que j'avais étudié en cours pendant le semestre,

en expliquant qu'il permettait de séparer clairement les couches sans ajouter la complexité des micro-services. Mon argument principal était simple : si l'endroit où on travaille n'est pas organisé, personne n'a envie d'y travailler, et chaque nouvelle fonctionnalité risquait d'aggraver le désordre. Les fondateurs ont validé cette proposition après discussion.

Le pattern que j'ai mis en place suit le schéma suivant : les fichiers de routes définissent les endpoints HTTP et redirigent vers les contrôleurs. Les contrôleurs récupèrent les paramètres de la requête, appellent les services et renvoient la réponse. Les services contiennent la logique métier (validations, calculs, règles) et utilisent Prisma pour interagir avec la base de données. Ce découpage a rendu le code beaucoup plus lisible et testable.

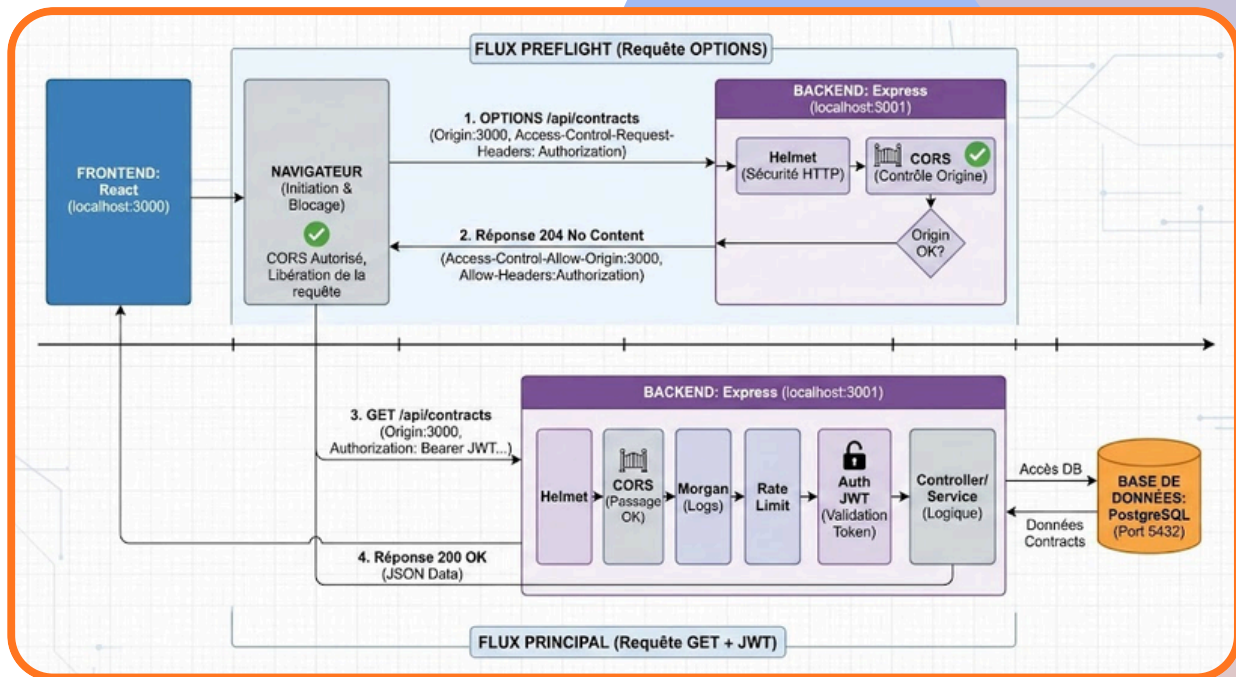


Figure 4 — Pattern MVC adopté pour le backend

Cette étape m’a permis de mobiliser directement une compétence du BUT : « Développer des applications informatiques », et plus précisément la conception d’architecture logicielle. C’était la première fois que je proposais une architecture sur un vrai projet, et le fait que ma proposition soit acceptée m’a donné confiance pour la suite.

Développement de nouvelles fonctionnalités backend

Une fois l’architecture en place, j’ai commencé à développer de nouveaux modules pour la plateforme. J’ai créé quatre modèles Prisma pour la base de données : Message pour la messagerie interne, Paiement pour les transactions financières, Avis pour les évaluations entre utilisateurs, et Milestone pour les jalons des contrats.

Pour chaque modèle, j’ai ensuite créé la triplète route / contrôleur / service en suivant le pattern MVC établi. Prenons l’exemple du module d’avis. Le service avis.service.ts contient trois méthodes principales. `getAvisByUser` récupère tous les avis reçus par un utilisateur. `createAvis` crée un nouvel avis après plusieurs vérifications : le contrat doit être terminé, l’utilisateur doit être partie prenante du contrat, il ne peut pas se noter lui-même, et il ne peut laisser qu’un seul avis par contrat. `getMoyenneUser` calcule la note moyenne d’un utilisateur.

Extrait — Vérification avant création d'un avis (avis.service.ts) :

```
async createAvis(data: {  
  contratId: number;  
  auteurId: number;  
  destinataireId: number;  
  noteGlobale: number;  
  noteQualite?: number;  
  noteDelais?: number;  
  noteCommunication?: number;  
  commentaire?: string;  
}) {  
  // Vérifier que le contrat est terminé  
  const contrat = await prisma.contrat.findUnique({  
    where: { id: data.contratId },  
  });  
  
  if (!contrat) {  
    throw new Error('Contrat non trouvé');  
  }  
  
  if (contrat.statut !== 'TERMINE') {  
    throw new Error('Le contrat doit être terminé pour laisser un avis');  
  }  
  
  // Vérifier que l'utilisateur ne se note pas lui-même  
  if (data.auteurId === data.destinataireId) {  
    throw new Error('Vous ne pouvez pas vous noter vous-même');  
  }  
}
```

Figure 5 — Extrait du service de gestion des avis

Le module des milestones était particulièrement intéressant car il reliait plusieurs concepts : un contrat peut avoir plusieurs jalons, chaque jalon représente une étape du chantier (Démolition , Installation électrique, Vérification finale....), et la validation d'un jalon peut déclencher automatiquement un paiement partiel. J'ai implémenté cette logique dans la méthode `validateMilestone` du service, qui vérifie que le jalon est bien terminé, le passe en statut « VALIDE » et crée un paiement si le montant est supérieur à zéro.

Toutes les routes créées sont protégées par un middleware d'authentification. Ce middleware vérifie la présence et la validité du token JWT dans le header `Authorization` de chaque requête. Si le token est absent ou invalide, la requête est rejetée avec une erreur 401. Ce mécanisme sécurise l'accès aux données personnelles des utilisateurs.

Au total, j'ai créé quatre fichiers de routes, quatre services et quatre contrôleurs, enregistrés dans le fichier principal `app.ts`. J'ai testé chaque endpoint avec Postman en simulant différents scénarios : requête valide, paramètre manquant, token invalide, ressource inexistante. Ces tests m'ont permis de corriger plusieurs bugs avant même que le frontend ne soit connecté.

Développement frontend : services API et composants React

Parallèlement au backend, j'ai travaillé sur le frontend pour connecter les interfaces utilisateur aux APIs que j'avais créées. La première étape a consisté à créer des fichiers de services API. Ces fichiers regroupent les fonctions qui appellent les endpoints du backend via la bibliothèque Axios. J'ai créé `paiementApi.js`, `avisApi.js`, `milestoneApi.js`, `favorisApi.js` et `notificationApi.js`, chacun contenant entre trois et cinq fonctions.

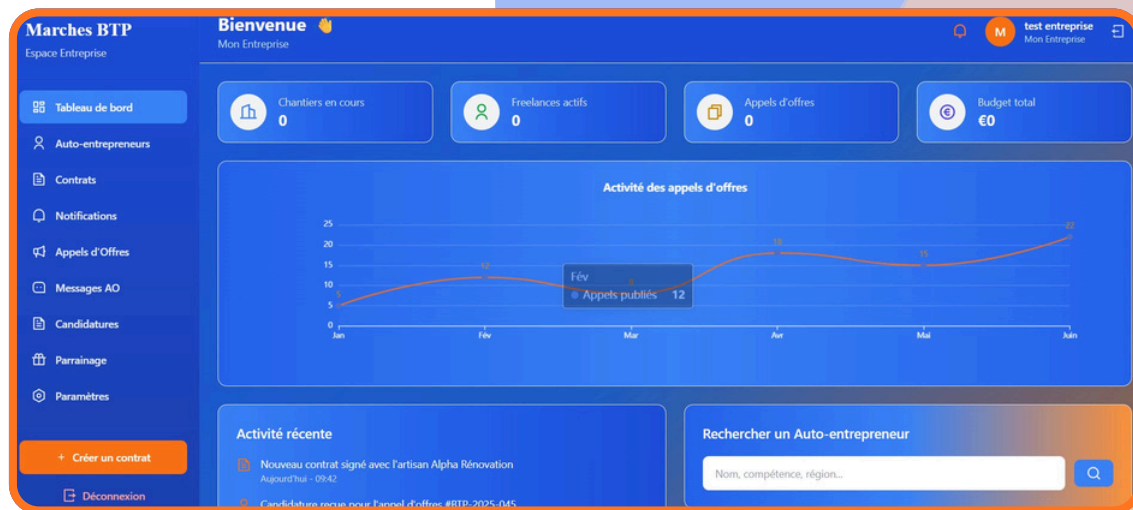


Figure 6 : Dashboard entreprise

Un composant que j'ai entièrement créé est `MesFavoris.jsx`, qui permet aux freelances de sauvegarder des appels d'offres intéressants. J'ai utilisé les hooks `useState` et `useEffect` pour gérer l'état, et j'ai implémenté un bouton « Retirer » sur chaque carte. Un bug m'a pris du temps : quand on retirait un favori, la carte restait affichée jusqu'au prochain rafraîchissement de la page. La solution était de mettre à jour le state local immédiatement avec un `filter()` sur le tableau des favoris, au lieu d'attendre un rechargement depuis l'API.

Adaptation responsive des interfaces

Une part importante de mon travail a concerné l'adaptation responsive des dashboards pour les écrans mobiles et tablettes. Nadia m'a d'abord montré comment utiliser les DevTools de Chrome (touche F12) pour simuler différentes tailles d'écran : 320px pour les petits smartphones, 375px pour les smartphones standards, 768px pour les tablettes et 1024px et plus pour les ordinateurs.

Le premier défi était la sidebar (menu latéral) des dashboards. Sur un écran d'ordinateur, elle est toujours visible sur le côté gauche. Mais sur mobile, elle prenait toute la largeur et masquait le contenu.

J'ai implémenté un menu « hamburger » : un bouton avec trois barres horizontales qui n'apparaît que sur mobile (grâce à la classe Tailwind `lg:hidden`). Au clic, la sidebar s'ouvre en glissant depuis la gauche avec une animation CSS (`translate-x`). Un overlay semi-transparent apparaît derrière pour fermer le menu en cliquant à côté. J'ai appliqué ce pattern sur les trois dashboards (entreprise, freelance, appels d'offres).

Le second défi concernait les tableaux de données. Les tableaux classiques avec colonnes sont illisibles sur mobile car ils débordent de l'écran. J'ai créé deux versions de chaque affichage : une version tableau visible uniquement sur tablette et plus (`hidden md:block`), et une version en cartes empilées visible uniquement sur mobile (`md:hidden`). Les cartes de statistiques en haut des dashboards utilisent une grille adaptative : une colonne sur mobile (`grid-cols-1`), deux sur tablette (`sm:grid-cols-2`) et quatre sur desktop (`lg:grid-cols-4`).

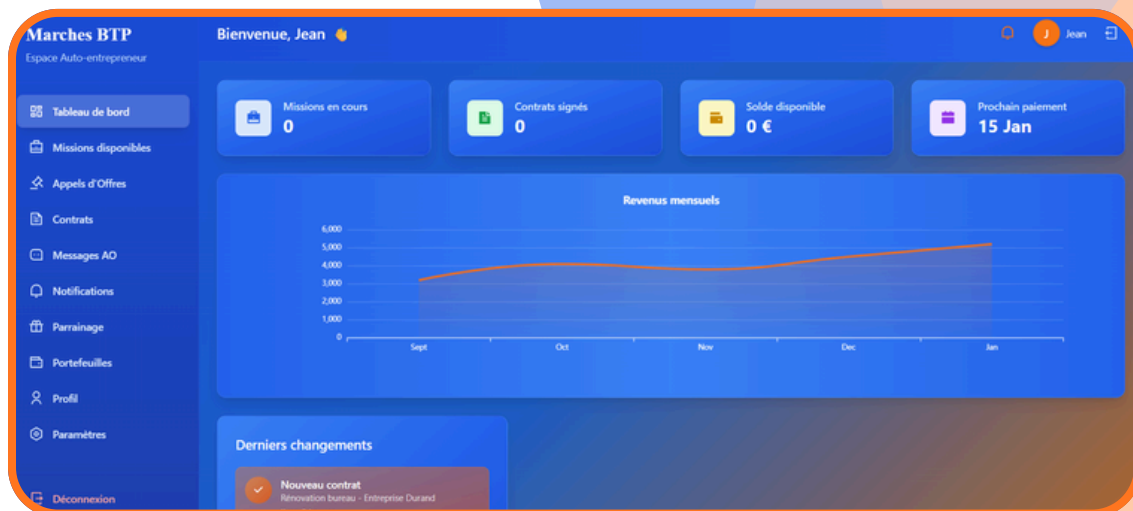


Figure 7 : Profil freelance sur pc

Figure 7.1 — Dashboard freelance en version desktop et mobile

Documentation Swagger de l'API REST

Les deux dernières semaines du stage ont été consacrées à la documentation de l'API avec le format OpenAPI 3.0, plus connu sous le nom de Swagger. L'objectif était de produire un fichier swagger.json décrivant l'ensemble des endpoints de l'API, pour que les futurs développeurs puissent comprendre comment utiliser chaque route sans avoir à lire le code source.

Comme je n'avais jamais utilisé Swagger, j'ai commencé par lire la documentation officielle sur swagger.io. J'ai appris la structure d'un fichier Swagger : la section info pour les métadonnées, servers pour l'URL du serveur, paths pour décrire chaque route, et components/schemas pour définir des structures réutilisables. J'ai également installé l'extension Swagger Viewer sur VS Code, ce qui me permettait de visualiser le rendu en temps réel pendant la rédaction.

Ma méthode de travail était systématique : pour chaque route, je la testais d'abord avec Postman en essayant différents scénarios (requête valide, paramètre manquant, token invalide), puis je documentais les paramètres d'entrée, le body attendu, les codes de réponse HTTP et les schémas de réponse avec des exemples concrets. J'ai organisé les routes par modules : authentification (11 routes), freelances (10 routes), entreprises (9 routes), appels d'offres (8 routes), contrats (11 routes), messages (7 routes), notifications (7 routes), géocodage (5 routes) et administration (10 routes).

La principale difficulté résidait dans les routes dont la réponse varie selon le rôle de l'utilisateur. Par exemple, la route GET /api/contrats ne renvoie pas les mêmes champs si l'utilisateur est un freelance ou une entreprise. J'ai résolu ce problème en utilisant des schémas communs dans components/schemas et en documentant les différences dans la description de chaque route.

Au final, le fichier swagger.json contient 77 opérations API documentées, réparties sur neuf modules. Ce document constitue un livrable durable pour l'entreprise : il permettra aux prochains développeurs de comprendre l'API sans avoir à fouiller dans le code source.

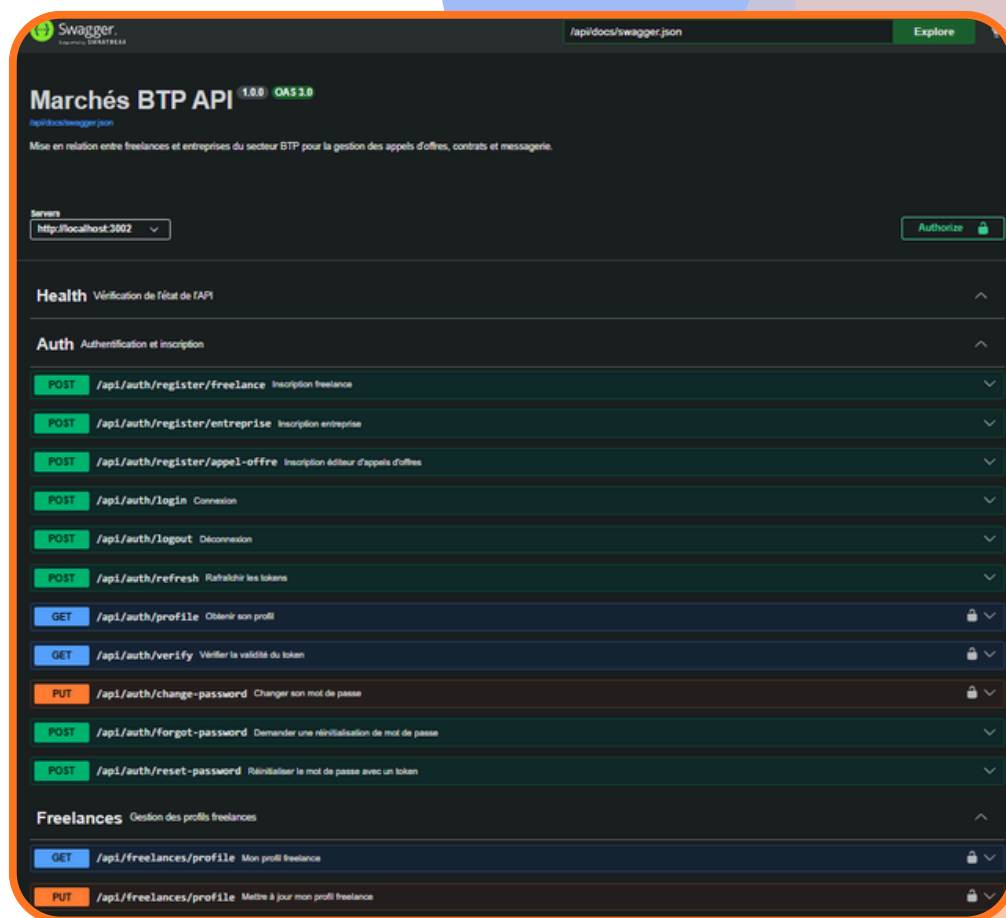
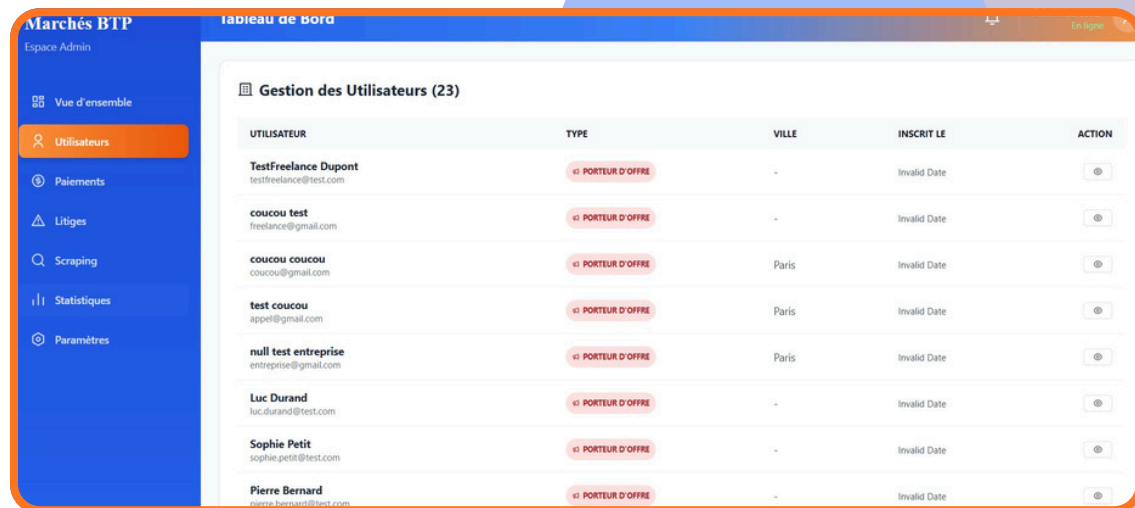


Figure 8 : Interface d'administration - Gestion des utilisateurs

Figure 8.1 — Aperçu de la documentation Swagger générée

Tests fonctionnels et rapport de bugs

En parallèle de la documentation, j'ai réalisé un test systématique de l'ensemble de la plateforme. J'ai navigué sur chaque page des quatre dashboards (administrateur, freelance, entreprise, appels d'offres) en notant tous les dysfonctionnements rencontrés. Pour chaque bug, j'ai pris une capture d'écran, décrit le problème observé, le comportement attendu, et suggéré le fichier à modifier pour le corriger.

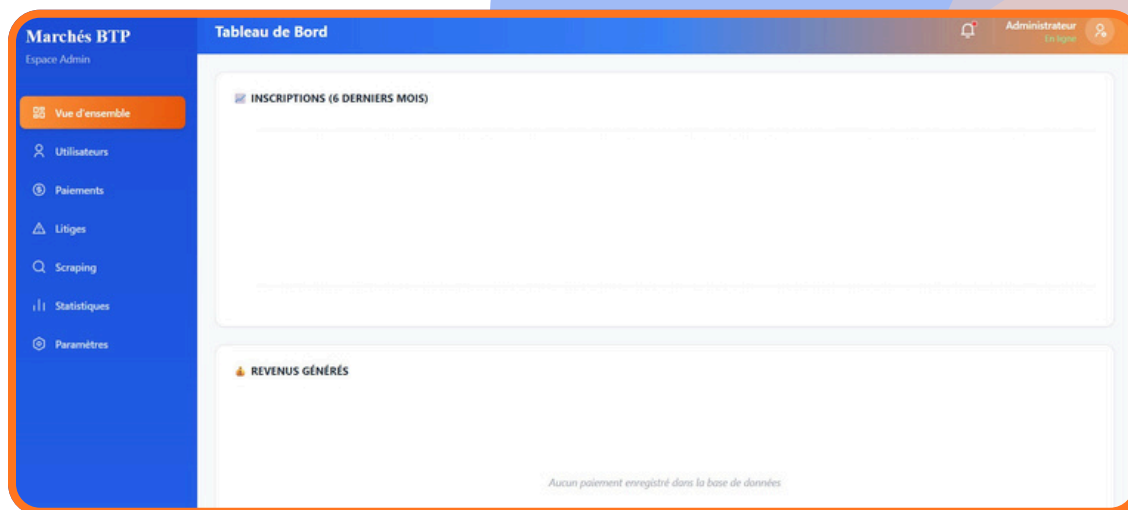


Figure 9 : Dashboard administrateur

Ce rapport, accompagné d'un tableau récapitulatif par fichier listant les actions à mener, constitue le deuxième livrable important de mon stage. Il permettra aux futurs développeurs de savoir exactement ce qui reste à corriger et dans quels fichiers intervenir.

Passation du projet

Le dernier jour de mon stage, j'ai présenté mes livrables à Marko et Brahim : la documentation Swagger et le rapport de tests. Marko a souligné que c'était la première fois que le projet disposait d'une documentation API complète, ce qui allait simplifier le travail des prochains intervenants.

Ensuite, il fallait leur transférer le projet complet. J'ai d'abord copié le dossier du projet sur une clé USB depuis mon PC Windows, mais quand nous avons voulu la brancher sur leur Mac, la clé était illisible. Le problème venait du format du système de fichiers : la clé, formatée en NTFS par Windows, n'est pas compatible en écriture avec macOS. Les fondateurs ont cherché une autre clé, mais je leur ai expliqué que le même problème se reproduirait. J'ai donc envoyé le projet par WeTransfer, téléchargé directement sur le Mac, puis copié sur une nouvelle clé USB formatée en exFAT, un format compatible avec les deux systèmes.

J'ai ensuite installé l'environnement de développement sur leur Mac : Node.js et PostgreSQL via Homebrew, création de la base de données, installation des dépendances npm, génération du client Prisma et exécution des migrations. J'ai ouvert deux terminaux — un pour le backend (npm run dev) et un pour le frontend (npm start) — et je leur ai montré comment lancer l'application. Tout fonctionnait. Ce dernier épisode a clôturé mon stage sur une note concrète : les fondateurs repartaient avec un projet fonctionnel, documenté et testé.

Bilan et compétences acquises

Compétences techniques

Ce stage m'a permis de progresser considérablement sur le plan technique. J'ai appris à utiliser les hooks (useState, useEffect) dans un contexte réel, à gérer l'état d'une application complète et à connecter des composants à une API REST. J'ai aussi découvert Tailwind CSS, que je ne connaissais pas du tout : son approche par classes utilitaires m'a paru déroutante au début, mais j'ai fini par l'apprécier pour sa rapidité, surtout pour le responsive.

Côté backend, j'avais quelques bases en Node.js et Express, mais le stage m'a permis de comprendre ce que signifie développer une API REST complète : définir des routes, implémenter des validations métier, gérer l'authentification avec JWT, et structurer le code selon un pattern clair. Prisma a été une découverte : cet ORM permet d'interagir avec la base de données en écrivant du TypeScript typé plutôt que des requêtes SQL brutes, ce qui rend le code plus lisible et plus sûr.

La documentation Swagger a été une compétence entièrement nouvelle. Je n'avais jamais entendu parler d'OpenAPI avant ce stage. Documenter 77 routes m'a demandé de la rigueur et de la patience, mais c'est un savoir-faire que je pourrai réutiliser dans n'importe quel projet professionnel.

Le tableau ci-dessous résume les compétences techniques mobilisées :

TECHNOLOGIE	NIVEAU AVANT LE STAGE	NIVEAU APRÈS LE STAGE
 React	 Débutant (cours)	Intermédiaire 
 Node.js / Express	Débutant	Intermédiaire 
 PostgreSQL	 Bases (cours)	Intermédiaire 
 Prisma	? Inconnu	Intermédiaire 
 Tailwind CSS	Inconnu	Intermédiaire 
 Swagger / OpenAPI	Inconnu	 Bonne maîtrise
 Postman	Débutant	 Bonne maîtrise
 JWT / Auth	 Notions (cours)	Intermédiaire 

Figure 10—Évolution des compétences techniques pendant le stage

Compétences professionnelles et humaines

Au-delà de la technique, ce stage m'a apporté beaucoup sur le plan professionnel. La compétence que j'ai le plus développée est sans doute la capacité à reprendre un code écrit par d'autres. À l'IUT, on travaille toujours sur nos propres projets, on connaît chaque ligne de code. Ici, j'ai dû comprendre la logique de développeurs que je n'ai jamais rencontrés, deviner leurs intentions à partir de noms de variables et de structures de fichiers. C'est un exercice frustrant au début, mais très formateur.

J'ai aussi appris à travailler en autonomie. Nadia était disponible pour répondre à mes questions, mais elle ne me donnait pas de solution toute faite : elle m'orientait vers la documentation ou me posait des questions pour que je trouve moi-même. Cette approche m'a poussé à chercher davantage avant de demander de l'aide, ce qui a renforcé ma capacité à résoudre des problèmes seul.

La communication avec les fondateurs m'a également appris à vulgariser des concepts techniques. Marko et Brahim n'étant pas développeurs, je devais expliquer mes choix sans utiliser de jargon technique. Par exemple, quand j'ai proposé l'architecture MVC, j'ai dû trouver des analogies simples pour justifier cette décision. Cet exercice de vulgarisation est une compétence que je sous-estimais avant le stage.

Enfin, l'organisation du temps entre présentiel et télétravail m'a demandé de la discipline. En télétravail, il n'y a personne pour vérifier si on travaille ou non : c'est une question de responsabilité personnelle.

J'ai appris à me fixer des objectifs de journée et à rester concentré même en travaillant depuis chez moi.

Difficultés rencontrées et leçons tirées

La difficulté principale a été la reprise du code existant. Le projet était volumineux, désorganisé, et je ne pouvais pas contacter les développeurs précédents pour leur poser des questions. J'ai dû reconstituer la logique métier uniquement à partir du code, en faisant des allers-retours entre le frontend et le backend. Cette situation m'a convaincu de l'importance de la documentation : j'ai vécu en tant que lecteur ce que les futurs développeurs auraient vécu sans ma documentation Swagger.

Une autre difficulté a été la polyvalence demandée. En startup, on ne reste pas cantonné à un seul domaine : j'ai fait du backend, du frontend, du responsive, de la documentation, des tests, de l'administration système (installation sur Mac). C'était parfois déstabilisant de changer de contexte, mais cela m'a permis d'avoir une vision complète du cycle de développement d'une application web.

Conclusion

Ce stage de huit semaines chez Marchés BTP a été une expérience très positive, à la fois sur le plan technique et humain. Je suis arrivé avec des connaissances théoriques apprises à l'IUT et j'en repars avec une vision concrète de ce que signifie développer un produit numérique dans un contexte professionnel.

Sur le plan technique, j'ai contribué à structurer le code selon un pattern MVC, créé de nouvelles fonctionnalités backend et frontend, adapté les interfaces pour les écrans mobiles, documenté l'intégralité des 77 routes de l'API et identifié 17 bugs à travers un rapport de tests détaillé. Ces livrables sont directement exploitables par les futurs développeurs qui reprendront le projet après mon départ.

Sur le plan humain, j'ai découvert le fonctionnement d'une startup, où la polyvalence et l'autonomie sont des qualités indispensables. J'ai appris à communiquer mes idées techniques à des interlocuteurs non-techniciens, à m'organiser entre présentiel et télétravail, et surtout à reprendre du code écrit par d'autres un exercice que je n'avais jamais pratiqué à l'IUT et qui constitue pourtant le quotidien de la plupart des développeurs en entreprise.

Ce stage m'a permis de mettre en pratique plusieurs compétences du BUT Informatique, notamment « Développer des applications informatiques » (à travers la conception d'architecture, le développement fullstack et le responsive), « Administrer des systèmes informatiques communicants » (à travers la documentation de l'API et la configuration d'environnements) et « Gérer des données de l'information » (à travers la modélisation avec Prisma et les tests fonctionnels).

Cette expérience confirme mon intérêt pour le développement web fullstack. J'envisage de poursuivre en BUT 3, idéalement en alternance, pour continuer à progresser dans un cadre professionnel tout en approfondissant mes connaissances académiques. Le stage chez Marchés BTP m'a montré que j'étais capable de m'adapter à un environnement nouveau, de prendre des initiatives et de mener à bien des missions concrètes : autant de qualités que je compte développer dans la suite de mon parcours.

Glossaire

API : Application Programming Interface. Interface permettant à deux applications de communiquer entre elles via des requêtes HTTP.

Axios : Bibliothèque JavaScript utilisée pour effectuer des requêtes HTTP depuis le frontend vers le backend.

Backend : Partie serveur d'une application, invisible pour l'utilisateur, qui gère la logique métier et l'accès aux données.

BTP : Bâtiment et Travaux Publics.

CRUD : Create, Read, Update, Delete. Les quatre opérations de base sur les données.

Endpoint : Point d'entrée d'une API, identifié par une URL et une méthode HTTP (GET, POST, etc.).

Express : Framework léger pour Node.js permettant de créer des serveurs web et des APIs REST.

Frontend : Partie client d'une application, visible par l'utilisateur dans le navigateur.

Hook (React) : Fonction spéciale de React (useState, useEffect) qui permet de gérer l'état et les effets dans les composants fonctionnels.

JWT : JSON Web Token. Standard pour l'authentification sans session, où un jeton signé est envoyé avec chaque requête.

Middleware : Fonction intermédiaire exécutée entre la réception d'une requête et l'envoi de la réponse (ex : vérification d'authentification).

MVC : Modèle-Vue-Contrôleur. Architecture qui sépare la logique métier, l'affichage et le contrôle des requêtes.

Node.js : Environnement d'exécution JavaScript côté serveur, utilisé pour développer des applications backend.

OpenAPI : Spécification standard pour décrire des APIs REST. Swagger est l'outil principal pour visualiser cette spécification.

ORM : Object-Relational Mapping. Outil qui permet de manipuler la base de données via du code objet plutôt que du SQL.

PostgreSQL : Système de gestion de bases de données relationnelles open source.

Prisma : ORM moderne pour Node.js et TypeScript, utilisé pour définir le schéma de la base et effectuer des requêtes.

React : Bibliothèque JavaScript développée par Meta pour construire des interfaces utilisateur.

Responsive : Conception web qui adapte l'affichage en fonction de la taille de l'écran (mobile, tablette, desktop).

REST : Representational State Transfer. Style d'architecture pour les APIs web basé sur les méthodes HTTP.

SAS : Société par Actions Simplifiée. Forme juridique courante pour les startups en France.

Swagger : Ensemble d'outils pour concevoir, documenter et tester des APIs REST.

Tailwind CSS : Framework CSS utilitaire qui fournit des classes prédéfinies pour styliser les éléments HTML.

Références

Documentation officielle React, Meta Platforms,
consultée en février 2026. <https://react.dev>

Documentation officielle Express.js, OpenJS Foundation,
consultée en février 2026. <https://expressjs.com>

Documentation Prisma ORM, Prisma Data Inc.,
consultée en février-mars 2026. <https://www.prisma.io/docs>

Documentation Tailwind CSS, Tailwind Labs,
consultée en février 2026. <https://tailwindcss.com/docs>

Spécification OpenAPI 3.0, SmartBear Software,
consultée en mars 2026. <https://swagger.io/specification>

Documentation PostgreSQL 15, The PostgreSQL Global Development Group,
consultée en janvier 2026. <https://www.postgresql.org/docs/15>

Documentation Node.js, OpenJS Foundation,
consultée en janvier 2026. <https://nodejs.org/docs>

Documentation Postman, Postman Inc.,
consultée en mars 2026. <https://learning.postman.com/docs>

Tutoriel Swagger UI, SmartBear Software,
consulté en mars 2026. <https://swagger.io/tools/swagger-ui>

Table des illustrations

Figure 1 — Espace de travail à l'incubateur IMT Business School

Figure 2 — Architecture technique de la plateforme Marchés BTP

Figure 3 — Pattern MVC adopté pour le backend

Figure 4 — Extrait du service de gestion des avis (avis.service.ts)

Figure 5 — Dashboard en version mobile (responsive)

Figure 6 — Interface de messagerie connectée à l'API

Figure 7 — Dashboard administrateur avec les bugs identifiés

Figure 8 — Aperçu de la documentation Swagger générée

Figure 9 — Évolution des compétences techniques pendant le stage

Figure 10 — Code du service de gestion des milestones

Figure 11 — Dashboard des appels d'offres

Figure 12 — Tutoriel Swagger UI utilisé pour apprendre le format OpenAPI

Annexes

Annexe A : Extrait de code — Service de gestion des milestones

```
marches-btp-backend-main > src > services > TS milestone.service.ts > ...
1  import { prisma } from '../config/database.js';
2
3  class MilestoneService {
4    /**
5     * Lister les jalons d'un contrat
6     */
7    async getMilestonesByContrat(contratId: number) {
8      return prisma.milestone.findMany({
9        where: { contratId },
10       orderBy: { ordre: 'asc' },
11     });
12   }
13
14   /**
15    * Créer un jalon (réservé à l'entreprise propriétaire du contrat)
16    */
17   async createMilestone(data: {
18     contratId: number;
19     titre: string;
20     description?: string;
21     montant?: number;
22     ordre: number;
23     dateEcheance?: Date;
24     userId: number;
25   }) {
26     const contrat = await prisma.contrat.findUnique({
27       where: { id: data.contratId },
28       include: { entreprise: true },
29     });
30
31     if (!contrat) {
32       throw new Error('Contrat non trouvé');
33     }
34
35     return prisma.milestone.create({
36       data: {
37         contratId: data.contratId,
38         titre: data.titre,
39         description: data.description || '',
40         montant: data.montant || 0,
41         ordre: data.ordre,
42         statut: 'EN_ATTENTE',
43         dateEcheance: data.dateEcheance || null,
44       },
45     });
46   }
47 }
```

```

48  /**
49   * Mettre à jour le statut d'un jalon
50   */
51  async updateStatus(milestoneId: number, statut: string) {
52    const milestone = await prisma.milestone.findUnique({
53      where: { id: milestoneId },
54    });
55
56    if (!milestone) {
57      throw new Error('Jalon non trouvé');
58    }
59
60    return prisma.milestone.update({
61      where: { id: milestoneId },
62      data: { statut: statut as any },
63    });
64  }
65
66  /**
67   * Valider un jalon terminé (peut déclencher un paiement)
68   */
69  async validateMilestone(milestoneId: number) {
70    const milestone = await prisma.milestone.findUnique({
71      where: { id: milestoneId },
72    });
73
74    if (!milestone) {
75      throw new Error('Jalon non trouvé');
76    }
77
78    if (milestone.statut !== 'TERMINE') {
79      throw new Error('Le jalon doit être terminé avant d\'être validé');
80    }
81
82    // Mettre à jour le statut à VALIDE
83    const updated = await prisma.milestone.update({
84      where: { id: milestoneId },
85      data: { statut: 'VALIDE' },
86    });
87
88    // Si le milestone a un montant, créer un paiement automatique
89    if (milestone.montant && Number(milestone.montant) > 0) {
90      await prisma.paiement.create({
91        data: {
92          contratId: milestone.contratId,
93          montant: milestone.montant,
94          type: 'MILESTONE',
95          description: `Paiement jalon : ${milestone.titre}`,
96          statut: 'EN_ATTENTE',
97        },
98      });
99    }
100
101    return updated;
102  }
103 }
104
105 export const milestoneService = new MilestoneService();
106 export default milestoneService;

```

Annexe B : Captures d'écran de la plateforme

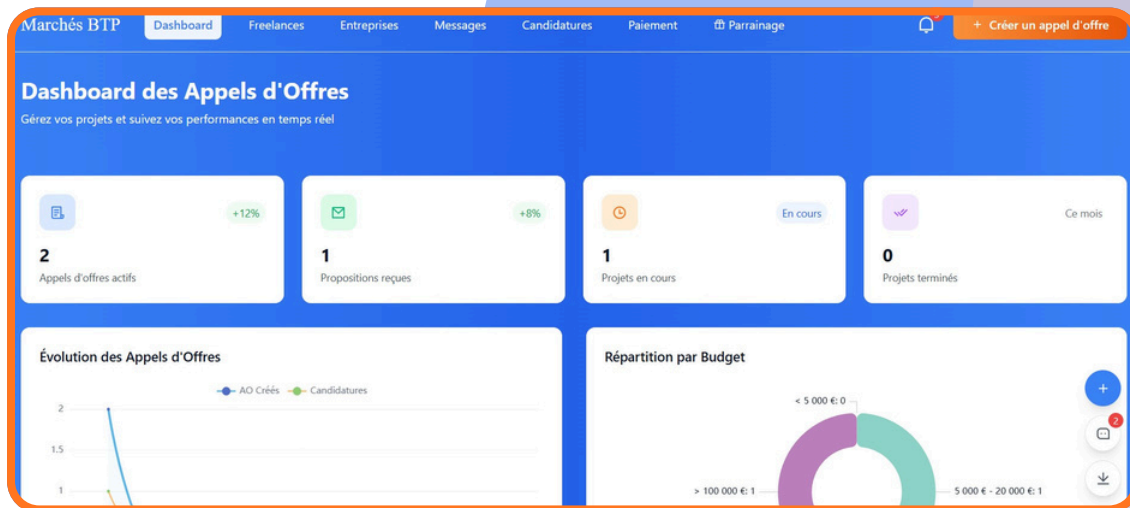


Figure 11 : Dashboard appels d'offres

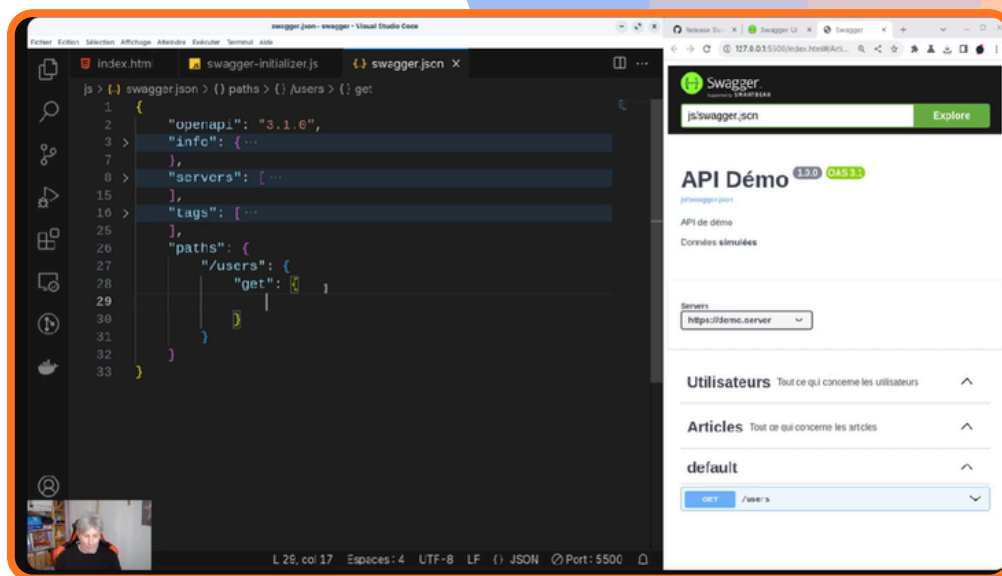


Figure 12 — Tutoriel Swagger UI utilisé pour apprendre le format OpenAPI